

Oracle — Data Contract (v1, frozen Jul 3)

For Sam-Charles (frontend) and Ibrahim (advisory layer)

This is the exact JSON shape every asset object will be delivered in. Build against this now — do not wait for the real model. Static mock files today, real files swapped in later, same shape, zero rework.

Single Asset Object

```
{
  "asset_id": "kiln_id_fan",
  "asset_name": "Kiln ID Fan",
  "plant": "Obajana",

  "tier": "amber",
  "risk_score": 62,

  "rul_estimate_days": 9,
  "rul_display": "≈9 days at current load",

  "pf_zone": 3,
  "pf_zone_label": "Accelerating Degradation",

  "driving_factors": [
    {
      "factor": "vibration_trend",
      "label": "Vibration Trend",
      "score": 4.2,
      "weight": 0.30,
      "trend": "rising",
      "detail": "+18% over 14 days"
    },
    {
      "factor": "maintenance_overdue",
      "label": "Maintenance Overdue",
      "score": 3.0,
      "weight": 0.25,
      "trend": "stable",
      "detail": "6 days past scheduled interval"
    }
  ]
}
```

```

    },
    {
      "factor": "failure_history",
      "label": "Historical Failure Frequency",
      "score": 2.5,
      "weight": 0.20,
      "trend": "stable",
      "detail": "2 failures in last 12 months"
    },
    {
      "factor": "load_vs_capacity",
      "label": "Load vs Rated Capacity",
      "score": 3.5,
      "weight": 0.15,
      "trend": "rising",
      "detail": "108% of rated load"
    },
    {
      "factor": "operator_anomalies",
      "label": "Operator-Reported Anomalies",
      "score": 1.0,
      "weight": 0.10,
      "trend": "stable",
      "detail": "1 report in last 7 days"
    }
  ],

  "trend_data": [
    { "day": 1, "vibration": 2.1, "temperature": 61.2, "current_draw": 88.0 },
    { "day": 2, "vibration": 2.2, "temperature": 61.5, "current_draw": 88.4 },
    { "day": 3, "vibration": 2.4, "temperature": 62.0, "current_draw": 89.1 }
  ],

  "advisory": {
    "recommendation": "Vibration trending +18% over 14 days with RUL estimated at",
    "urgency": "high",
    "suggested_action_by": "2026-07-12"
  },

  "escalation": {
    "status": "flagged_for_inspection",
    "queue": "amber_queue",
    "notify": "maintenance_planner"
  },

  "last_updated": "2026-07-03T10:00:00Z"
}

```

Field Reference

Field	Type	Notes
<code>asset_id</code>	string	lowercase_snake_case, stable identifier, never changes once set
<code>asset_name</code>	string	display name for UI
<code>plant</code>	string	for the optional multi-plant view
<code>tier</code>	string	one of: "green" , "amber" , "red" — always lowercase, this drives UI color directly
<code>risk_score</code>	number	0–100
<code>rul_estimate_days</code>	number	raw number, for any calculations/sorting Sam-Charles needs
<code>rul_display</code>	string	pre-formatted for direct display, so frontend never has to build this string itself
<code>pf_zone</code>	number	1–4, matches the four P-F zones
<code>pf_zone_label</code>	string	matches zone 1–4 to its name, for direct display
<code>driving_factors</code>	array	always 5 objects, same 5 factors, same order, every asset
<code>driving_factors[].score</code>	number	0–5
<code>driving_factors[].weight</code>	number	0–1, matches Section 4 weights, constant across all assets
<code>driving_factors[].trend</code>	string	one of: "rising" , "falling" , "stable"
<code>trend_data</code>	array	one object per day; length will grow as simulated data accumulates — frontend should handle variable length, not assume a fixed count
<code>advisory</code>	object	this whole object is Ibrahim's output — his layer fills this in, everything above it is yours

<code>advisory.urgency</code>	string	one of: "low", "medium", "high" — separate from <code>tier</code> , since urgency reflects RUL/time pressure specifically
<code>escalation.status</code>	string	one of: "logged_only", "flagged_for_inspection", "work_order_generated"
<code>escalation.notify</code>	string	one of: "none", "maintenance_planner", "maintenance_manager"

Rules Everyone Follows

1. **Never rename a field without telling the other two people.** If something needs to change, flag it in the group chat before changing it — a silent rename breaks the other person's build with no warning.
 2. `tier`, `pf_zone`, `trend`, `urgency`, `escalation.status` **are closed vocabularies** — only the exact string values listed above. Don't introduce a new one (e.g. don't add "orange" as a tier) without updating this doc first.
 3. **Three assets, three files.** Deliver as `kiln_id_fan.json`, `raw_mill.json`, `cement_mill.json` — one asset object per file, matching the shape above exactly.
 4. **Sam-Charles builds against this today** using 3 hand-written mock files in this exact shape. When you (Goodluck) and Ibrahim finish your real pipeline, you output files in the same shape and just replace the mocks — nothing in his frontend code should need to change.
 5. **Ibrahim can also build against this today** — he can mock the top-level fields (`tier`, `risk_score`, `driving_factors`, `trend_data`) himself to test his advisory logic in isolation, without waiting for your real scoring engine. He only needs the real numbers once you're generating them for real — the shape doesn't change either way.
-

Suggested Mock Files to Hand Sam-Charles Today

Send him 3 files like the example above, one per flagship asset:

- `kiln_id_fan.json` — set as **amber**, mid-crisis, most dramatic (this is your flagship demo asset)

- `raw_mill.json` — set as **green**, healthy, boring on purpose (shows contrast)
- `cement_mill.json` — set as **red**, worst case (shows the full escalation path in the demo)

Having one of each tier in the mock data means Sam-Charles's UI gets tested against all three colors/states from day one, instead of everything looking the same until real data shows up later.